

Lesson 6: Eloquent ORM - Basics

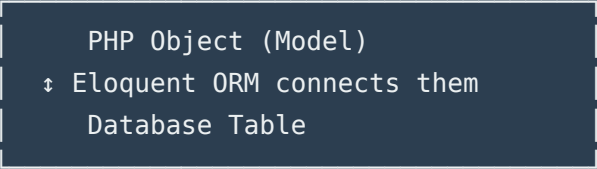
□ Contents

1. [Introduction to Eloquent ORM](#)
 2. [Creating Models](#)
 3. [Retrieving Data](#)
 4. [Creating Records](#)
 5. [Updating Records](#)
 6. [Deleting Records](#)
 7. [Soft Deletes](#)
 8. [Query Scopes](#)
 9. [Accessors & Mutators](#)
 10. [Mass Assignment](#)
 11. [Timestamps](#)
 12. [Practical Examples](#)
-

Introduction to Eloquent ORM

What is ORM?

ORM stands for **Object-Relational Mapping**



```
graph TD; A[PHP Object (Model)] <--> B[Eloquent ORM connects them]; B <--> C[Database Table];
```

PHP Object (Model)
↕ Eloquent ORM connects them
Database Table

Why Eloquent?

□ Write SQL queries in an easy and clear way □ Work with data as Objects □ Automatic protection from SQL Injection □ Powerful relationships between tables □ Save time and effort

Traditional vs Eloquent

Traditional SQL:

```
$results = DB::select('SELECT * FROM users WHERE active = ?', [1]);
foreach ($results as $row) {
    echo $row->name;
}
```

Eloquent Way:

```
$users = User::where('active', 1)->get();
foreach ($users as $user) {
    echo $user->name;
}
```

Creating Models

Create Simple Model

```
php artisan make:model Product
```

File created at: app/Models/Product.php

```
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Model;

class Product extends Model
{
    //
}
```

Create Model with Migration

```
php artisan make:model Product -m
```

Create Everything at Once

```
php artisan make:model Product -mfsc
```

Options:

- `-m`: Migration
- `-f`: Factory
- `-s`: Seeder
- `-c`: Controller

Basic Model Properties

```
class Product extends Model
{
    // Table name (optional - Laravel determines automatically)
    protected $table = 'products';

    // Primary key
    protected $primaryKey = 'id';

    // Key type
    protected $keyType = 'int';

    // Is key auto-incrementing?
    public $incrementing = true;

    // Use Timestamps?
    public $timestamps = true;

    // Customize Timestamps names
    const CREATED_AT = 'created_at';
    const UPDATED_AT = 'updated_at';
}
```

Retrieving Data

1. Retrieve All Records

```
// All products
$products = Product::all();

// All products with specific columns
$products = Product::all(['name', 'price']);
```

2. Find by Primary Key

```
// Find product with ID 1
$product = Product::find(1);

// Find multiple products
$products = Product::find([1, 2, 3]);

// Find or throw exception
$product = Product::findOrFail(1);
```

3. Where Queries

```
// Single condition
$products = Product::where('price', '>', 100)->get();

// Multiple conditions
$products = Product::where('category', 'Electronics')
    ->where('price', '<', 1000)
    ->get();

// OR condition
$products = Product::where('category', 'Electronics')
    ->orWhere('category', 'Computers')
    ->get();

// WHERE IN
$products = Product::whereIn('id', [1, 2, 3])->get();

// WHERE BETWEEN
$products = Product::whereBetween('price', [100, 500])->get();

// WHERE NULL
$products = Product::whereNull('deleted_at')->get();

// WHERE DATE
$products = Product::whereDate('created_at', '2024-01-01')->get();
```

4. Ordering and Limiting

```
// Ordering
$products = Product::orderBy('price', 'desc')->get();
$products = Product::orderBy('name')->get(); // ascending by default

// Latest and Oldest
$products = Product::latest()->get(); // by created_at
$products = Product::oldest()->get();

// Limit results
$products = Product::take(10)->get();
$products = Product::limit(10)->get();

// Skip and take
$products = Product::skip(10)->take(5)->get();
```

5. Retrieve Single Record

```
// First record
$product = Product::first();

// First with condition
$product = Product::where('active', 1)->first();

// First or throw exception
$product = Product::where('slug', 'laptop')->firstOrFail();

// First or create new
$product = Product::firstOrCreate(
    ['slug' => 'laptop'],
    ['name' => 'Laptop', 'price' => 5000]
);

// First or return new Model (without saving)
$product = Product::firstOrCreate(['slug' => 'laptop']);
```

6. Count and Aggregates

```
// Count records
$count = Product::count();
$count = Product::where('active', 1)->count();

// Max and min
```

```
$max = Product::max('price');
$min = Product::min('price');

// Sum and average
$sum = Product::sum('price');
$avg = Product::avg('price');
```

7. Chunking (for large datasets)

```
// Process data in batches
Product::chunk(100, function ($products) {
    foreach ($products as $product) {
        // Process each product
    }
});

// Lazy Loading
Product::lazy()->each(function ($product) {
    // Process each product
});
```

Creating Records

1. create() Method

```
// Create and save directly
$product = Product::create([
    'name' => 'Laptop',
    'slug' => 'laptop',
    'price' => 5000,
    'stock' => 10,
]);

// Must define fillable fields in Model:
protected $fillable = ['name', 'slug', 'price', 'stock'];
```

2. save() Method

```
// Create new instance
$product = new Product;
```

```
$product->name = 'Laptop';  
$product->slug = 'laptop';  
$product->price = 5000;  
$product->stock = 10;  
$product->save();
```

3. Create Multiple Records

```
Product::insert([  
    ['name' => 'Product 1', 'price' => 100],  
    ['name' => 'Product 2', 'price' => 200],  
    ['name' => 'Product 3', 'price' => 300],  
]);  
  
// Note: insert() doesn't use $fillable and doesn't add timestamps
```

4. firstOrCreate & updateOrCreate

```
// Find or create  
$product = Product::firstOrCreate(  
    ['slug' => 'laptop'],           // Search criteria  
    ['name' => 'Laptop', 'price' => 5000] // Data for creation  
);  
  
// Update or create  
$product = Product::updateOrCreate(  
    ['slug' => 'laptop'],  
    ['name' => 'Gaming Laptop', 'price' => 7000]  
);
```

Updating Records

1. Update Single Record

```
// Find then update  
$product = Product::find(1);  
$product->price = 6000;  
$product->save();  
  
// Direct update
```

```
$product = Product::find(1);  
$product->update(['price' => 6000]);
```

2. Update Multiple Records

```
// Update all products  
Product::where('category', 'Electronics')  
    ->update(['discount' => 10]);
```

3. Increment & Decrement

```
// Increment  
$product = Product::find(1);  
$product->increment('views');           // +1  
$product->increment('views', 5);        // +5  
  
// Decrement  
$product->decrement('stock');           // -1  
$product->decrement('stock', 2);        // -2  
  
// With other fields  
$product->increment('orders', 1, ['last_ordered_at' => now()]);
```

Deleting Records

1. Delete Single Record

```
// Find then delete  
$product = Product::find(1);  
$product->delete();  
  
// Direct delete  
Product::destroy(1);  
  
// Delete multiple records  
Product::destroy([1, 2, 3]);  
Product::destroy(1, 2, 3);
```


2. Delete with Condition

```
// Delete all inactive products
Product::where('active', 0)->delete();
```

3. Delete All Records

```
// Delete all (dangerous!)
Product::truncate();
```

Soft Deletes

What is Soft Delete?

Soft Delete = Doesn't actually delete the record, just sets a date in `deleted_at` field

Setup Soft Deletes

1. In Migration:

```
Schema::create('products', function (Blueprint $table) {
    $table->id();
    $table->string('name');
    $table->decimal('price', 10, 2);
    $table->timestamps();
    $table->softDeletes(); // Adds deleted_at column
});
```

2. In Model:

```
use Illuminate\Database\Eloquent\SoftDeletes;

class Product extends Model
{
    use SoftDeletes;
}
```

Using Soft Deletes

```
// Soft delete
$product = Product::find(1);
$product->delete(); // Sets date in deleted_at

// Retrieve deleted records
$products = Product::withTrashed()->get();

// Only deleted
$products = Product::onlyTrashed()->get();

// Restore deleted record
$product = Product::withTrashed()->find(1);
$product->restore();

// Restore multiple records
Product::onlyTrashed()
    ->where('category', 'Old')
    ->restore();

// Permanent delete
$product = Product::withTrashed()->find(1);
$product->forceDelete();

// Check if deleted
if ($product->trashed()) {
    echo "Deleted";
}
```

Query Scopes

What is a Scope?

Scope = Reusable query in Model

Local Scopes

```
class Product extends Model
{
    // Scope for active products
    public function scopeActive($query)
```

```

{
    return $query->where('active', 1);
}

// Scope for cheap products
public function scopeCheap($query, $maxPrice = 100)
{
    return $query->where('price', '<=', $maxPrice);
}

// Scope for best sellers
public function scopePopular($query)
{
    return $query->where('sales', '>', 100)
        ->orderBy('sales', 'desc');
}
}

```

Usage:

```

// Active products
$products = Product::active()->get();

// Active and cheap
$products = Product::active()->cheap(50)->get();

// Popular products
$products = Product::popular()->take(10)->get();

// Combine Scopes
$products = Product::active()
    ->cheap(200)
    ->orderBy('name')
    ->get();

```

Global Scopes

```

// In Model
protected static function booted()
{
    static::addGlobalScope('active', function ($query) {
        $query->where('active', 1);
    });
}

```

```
// Now all queries will include this condition automatically
$products = Product::all(); // Only active

// Ignore Global Scope
$products = Product::withoutGlobalScope('active')->get();
```

Accessors & Mutators

Accessors (Reading)

Accessor = Modify value when reading

```
class Product extends Model
{
    // Accessor for name
    protected function name(): Attribute
    {
        return Attribute::make(
            get: fn ($value) => ucfirst($value), // Capitalize first
        );
    }

    // Accessor for price with currency
    protected function price(): Attribute
    {
        return Attribute::make(
            get: fn ($value) => number_format($value, 2) . ' SAR',
        );
    }

    // New accessor (not in database)
    protected function fullName(): Attribute
    {
        return Attribute::make(
            get: fn () => "{$this->name} - {$this->category}",
        );
    }
}
```

Usage:

```
$product = Product::find(1);
```

```
echo $product->name;      // Laptop (capitalized automatically)
echo $product->price;      // 5,000.00 SAR
echo $product->full_name; // Laptop - Electronics
```

Mutators (Writing)

Mutator = Modify value when saving

```
class Product extends Model
{
    // Mutator for name
    protected function name(): Attribute
    {
        return Attribute::make(
            set: fn ($value) => strtolower($value), // Save in lowercase
        );
    }

    // Mutator for price (remove currency)
    protected function price(): Attribute
    {
        return Attribute::make(
            set: fn ($value) => (float) str_replace(',', '', $value),
        );
    }
}
```

Usage:

```
$product = new Product;
$product->name = 'LAPTOP'; // Saved as: laptop
$product->price = '5,000'; // Saved as: 5000
$product->save();
```

Accessor & Mutator Together

```
protected function price(): Attribute
{
    return Attribute::make(
        get: fn ($value) => number_format($value, 2) . ' SAR',
        set: fn ($value) => (float) str_replace(',', '', $value),
    );
}
```

```
}
```

Mass Assignment

What is Mass Assignment?

Create/update multiple fields at once:

```
Product::create([
    'name' => 'Laptop',
    'price' => 5000,
    'stock' => 10,
]);
```

Protection: fillable vs guarded

1. \$fillable (Allow):

```
class Product extends Model
{
    // Only allowed fields
    protected $fillable = [
        'name',
        'slug',
        'price',
        'stock',
    ];
}
```

2. \$guarded (Block):

```
class Product extends Model
{
    // Only blocked fields
    protected $guarded = [
        'id',
        'created_at',
        'updated_at',
    ];
}
```

```
// Or allow all fields (unsafe!)
protected $guarded = [];

}
```

Why Protection Matters?

```
// Without protection, user can:
Product::create($request->all());

// If user sends:
// ['name' => 'Laptop', 'price' => 1, 'is_admin' => 1]
// is_admin will be saved too!
```

Timestamps

Default Timestamps

```
class Product extends Model
{
    public $timestamps = true; // Default

    // Laravel adds automatically:
    // created_at
    // updated_at
}
```

Customize Timestamps

```
class Product extends Model
{
    // Change names
    const CREATED_AT = 'creation_date';
    const UPDATED_AT = 'last_update';

    // Or disable Timestamps
    public $timestamps = false;
}
```

Control Timestamps

```
// Save without updating updated_at
$product->saveQuietly();

// Or
$product->timestamps = false;
$product->save();

// Update Timestamps manually
$product->touch();

// Update updated_at only
$product->update(['updated_at' => now()]);
```

Practical Examples

Example 1: Product System

```
// Create new product
$product = Product::create([
    'name' => 'Gaming Laptop',
    'slug' => 'gaming-laptop',
    'description' => 'High performance laptop',
    'price' => 8000,
    'stock' => 5,
    'category' => 'Electronics',
]);

// Show Electronics only
$electronics = Product::where('category', 'Electronics')
    ->active()
    ->orderBy('price', 'desc')
    ->get();

// Update stock after order
$product = Product::find(1);
$product->decrement('stock');
$product->increment('sales');

// Delete old products
Product::where('stock', 0)
```



```
->where('updated_at', '<', now()->subMonths(6))
->delete();
```

Example 2: Blog System

```
// Create post
$post = Post::create([
    'title' => 'Learn Laravel',
    'slug' => 'learn-laravel',
    'content' => 'Full content here...',
    'user_id' => auth()->id(),
    'published_at' => now(),
]);

// Show published posts only
$posts = Post::where('published_at', '<=', now())
    ->orderBy('published_at', 'desc')
    ->paginate(10);

// Increment views
$post->increment('views');

// Popular posts
$popular = Post::where('views', '>', 1000)
    ->orderBy('views', 'desc')
    ->take(5)
    ->get();
```

Example 3: User System

```
// Create user
$user = User::create([
    'name' => 'Ahmed',
    'email' => 'ahmed@example.com',
    'password' => Hash::make('password'),
]);

// Find by email
$user = User::where('email', 'ahmed@example.com')->first();

// Update profile
$user->update([
    'name' => 'Ahmed Ali',
    'phone' => '0501234567',
]);
```

```
// Verify user
$user->email_verified_at = now();
$user->save();
```

Important Tips

📦 Best Practices

1. Use Eloquent instead of raw SQL:

```
// ❌ Bad
DB::select('SELECT * FROM products WHERE active = 1');
```

```
// ✅ Good Product::where('active', 1)->get();
```

2. **Use Query Scopes:**

```
```php
```

```
// ❌ Code repetition
```

```
Product::where('active', 1)->where('price', '<', 100)->get();
```

```
Product::where('active', 1)->orderBy('name')->get();
```

```
// ✅ Reusable Scope
```

```
Product::active()->cheap(100)->get();
```

```
Product::active()->orderBy('name')->get();
```

#### 3. Use Accessors for display only:

```
// ✅ Correct
protected function formattedPrice(): Attribute
{
 return Attribute::make(
 get: fn ($value) => number_format($this->price, 2),
);
}
```

#### 4. Protect data with fillable/guarded:

```
// ☐ Safe
protected $fillable = ['name', 'email'];
User::create($request->validated());
```

## ⚠ Common Mistakes

### 1. Forgetting fillable:

```
// ☐ Error: MassAssignmentException
Product::create(['name' => 'Test']);
```

// ☐ Correct: Define fillable in Model protected \$fillable = ['name'];

```
2. **Using all() for large datasets:**
```php
// ☐ Bad: Memory intensive
$products = Product::all(); // 100,000 records!

// ☐ Good: Use chunk or pagination
Product::chunk(100, function ($products) {
    // Process
});
```

3. Forgetting withTrashed():

```
// ☐ Won't find deleted record
$product = Product::find(1);
```

// ☐ Correct \$product = Product::withTrashed()->find(1);

Next Step

After completing this lesson, you're ready for:

****Lesson 7****: Eloquent Relationships
- One to One

- One to Many
- Many to Many
- Polymorphic Relations

****Happy Learning! ☺****